

Inference: Three-Layer Framework

Inference Engineering — inferenceengineering.tech

The Three Layers

Runtime:

Execution of the model: kernels, attention, KV cache, batching, quantization. Happens on GPU.

Infrastructure:

Servers, networking, autoscaling, load balancing, multi-cloud capacity. Happens on machines.

Tooling:

APIs, SDKs, observability, routing, caching at application level. Happens in software.

Key Terms

Inference:

Running a trained model on new inputs to produce outputs — not training.

Prefill:

Processing the entire input prompt in parallel. Compute-bound. One forward pass.

Decode:

Auto-regressive token generation one token at a time. Memory-bandwidth-bound.

TTFT:

Time to First Token — measures prefill latency. Critical for interactive apps.

TPOT:

Time Per Output Token — measures decode speed. Drives streaming UX.

Throughput:

Tokens generated per second across all concurrent requests. Measures hardware utilization.

Runtime / Infra / Tooling Split

Layer	Concern	Example Tools
Runtime	Kernel efficiency, batching	vLLM, TensorRT-LLM, SGLang
Infrastructure	Scaling, placement, cost	Kubernetes, Karpenter, Ray
Tooling	Routing, caching, evals	LiteLLM, Prometheus, Langfuse

Inference Lifecycle Steps

- ☐ Request arrives at load balancer
- ☐ Router selects backend instance
- ☐ KV cache checked for prefix reuse
- ☐ Prefill: prompt tokens processed in parallel
- ☐ Decode: tokens generated auto-regressively
- ☐ Response streamed back to client
- ☐ Metrics emitted to observability stack

Prerequisites: Latency, Throughput & Budgets

Inference Engineering — inferenceengineering.tech

Core Formulas

End-to-end latency:

$$\text{latency} = \text{TTFT} + (\text{output_tokens} \times \text{TPOT})$$

Throughput:

$$\text{throughput} = \text{total_output_tokens} / \text{wall_clock_seconds}$$

Cost per token:

$$\text{cost_per_token} = \text{instance_cost_per_hour} / (\text{throughput} \times 3600)$$

Concurrency:

$$\text{concurrency} = \text{throughput} \times \text{avg_latency} \quad [\text{Little's Law}]$$

GPU utilization:

$$\text{MFU} = \text{achieved_FLOPS} / \text{peak_FLOPS} \quad [\text{Model FLOP Utilization}]$$

Latency Budget Targets

Use Case	TTFT Target	TPOT Target
Interactive chat	< 500 ms	< 30 ms
Streaming assistant	< 1 s	< 50 ms
Batch summarization	< 10 s	< 100 ms
Offline processing	no target	maximize throughput

Key Definitions

P50 / P95 / P99:

Percentile latencies. P99 = worst 1 in 100 requests. Use P99 for SLA commitments.

SLO:

Service Level Objective — internal target (e.g., P99 TTFT < 1s).

SLA:

Service Level Agreement — contractual commitment to customers.

Cold start:

Latency penalty when a model is not loaded in GPU memory. Can be 10-60s.

Warm pool:

Pre-loaded instances kept idle to avoid cold starts at the cost of idle GPU spend.

Model Selection Checklist

- ☐ Define quality metric (MMLU, MT-Bench, task-specific eval)
- ☐ Set latency budget before picking model size
- ☐ Benchmark on YOUR data — leaderboards are proxies
- ☐ Check context length requirements
- ☐ Estimate VRAM: $\text{model_params_B} \times \text{bytes_per_param} + \text{KV cache}$
- ☐ Consider fine-tuning smaller model vs. prompting larger one

Models: Transformer Architecture

Inference Engineering — inferenceengineering.tech

Core Formulas

Scaled dot-product attention:

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

Model parameters (approx):

$$\text{params} \approx H \times 12 \times n_{\text{layers}} \times d_{\text{model}}^2 \quad (\text{dominant term})$$

VRAM for weights (FP16):

$$\text{VRAM}_{\text{weights}} = \text{params} \times 2 \text{ bytes}$$

KV cache per token:

$$\text{KV_bytes} = 2 \times n_{\text{layers}} \times d_{\text{head}} \times n_{\text{heads}} \times \text{bytes_per_element}$$

Arithmetic intensity (decode):

$$AI = \text{FLOPs} / \text{memory_bytes} \approx 1 \text{ op/byte for small batch}$$

Prefill vs. Decode

Property	Prefill	Decode
Parallelism	All tokens at once	One token at a time
Bottleneck	Compute (FLOP-bound)	Memory bandwidth (BW-bound)
Batch size effect	Linear throughput gain	Diminishing returns
KV cache	Written	Read + appended
Latency metric	TTFT	TPOT

Transformer Block Components

Multi-Head Attention (MHA):

Full Q/K/V for all heads. Most memory-intensive. $n_{\text{heads}} \times d_{\text{head}} = d_{\text{model}}$.

Grouped Query Attention (GQA):

Multiple query heads share fewer K/V heads. Reduces KV cache size. Used in Llama 3, Mistral.

Multi-Query Attention (MQA):

All query heads share a single K/V head. Maximum KV compression. Used in Falcon.

Feed-Forward Network (FFN):

Two linear layers with activation. Typically 4× hidden dim. SwiGLU uses 3 matrices.

RMSNorm:

Root Mean Square normalization. Cheaper than LayerNorm. Used in most modern LLMs.

MoE vs Dense

Dense model:

All parameters active per token. Compute cost scales with total params.

Mixture of Experts (MoE):

Router selects top-K experts per token. Active params $\ll$ total params. Higher VRAM, lower FLOPs.

Expert routing:

Token routed to K of N experts (typically K=2, N=8 or N=64). Load balancing is critical.

Hardware: GPUs & Accelerators

Inference Engineering — inferenceengineering.tech

GPU Specs Reference

GPU	Architecture	HBM (GB)	Mem BW (TB/s)	FP16 TFLOPS	TDP (W)
H100 SXM5	Hopper	80	3.35	989	700
H200 SXM5	Hopper	141	4.8	989	700
B200 SXM6	Blackwell	192	8.0	2250	1000
B300 SXM6	Blackwell Ultra	288	8.0	2500	1000
A100 SXM4	Ampere	80	2.0	312	400
L4	Ada Lovelace	24	0.3	121	72

Memory Hierarchy

Level	Capacity	Bandwidth	Latency
Registers	~256 KB/SM	~100 TB/s	~1 cycle
L1 / Shared Mem	128–256 KB/SM	~19 TB/s	~20 cycles
L2 Cache	50–96 MB	~7 TB/s	~200 cycles
HBM (VRAM)	80–288 GB	3–8 TB/s	~600 cycles
NVLink	multi-GPU	0.9 TB/s	~ 1 ¼ s
PCIe 5.0	CPU-GPU	0.128 TB/s	~ 5 ¼ s

Key Hardware Concepts

Arithmetic Intensity:

FLOPs / bytes moved. If AI < ops:byte ratio of GPU, workload is memory-bandwidth-bound.

Roofline model:

Performance = min(peak_FLOPS, bandwidth × AI). Identifies whether kernel is compute or BW limited.

NVLink:

NVIDIA's high-speed GPU interconnect. 900 GB/s (NVLink 4.0). Used for tensor parallelism.

NVSwitch:

All-to-all NVLink switching fabric in DGX/HGX nodes. Enables full-bandwidth GPU mesh.

MIG:

Multi-Instance GPU. Partition one H100 into up to 7 isolated GPU instances. Great for small models.

Sizing Checklist

- ☐ Calculate model VRAM: $\text{params} \times \text{bytes_per_param}$ (FP16=2, INT8=1, FP8=1, INT4=0.5)
- ☐ Add KV cache: $2 \times \text{layers} \times \text{heads} \times \text{d_head} \times \text{context_len} \times \text{batch} \times \text{dtype_bytes}$
- ☐ Add activation memory: ~1 GB overhead per GPU
- ☐ Check NVLink topology if using tensor parallelism
- ☐ Verify PCIe bandwidth for CPU-GPU data transfers
- ☐ Consider MIG for small models to reduce cost

Software: Stack & Inference Engines

Inference Engineering — inferenceengineering.tech

Software Stack Layers

Layer	What It Does	Examples
Application	API, routing, auth, caching	FastAPI, LiteLLM, nginx
Inference Engine	Batching, KV cache mgmt, sched	vLLM, SGLang, TRT-LLM
DL Framework	Tensor ops, autograd, model io	PyTorch, JAX
Kernel Libraries	Optimized GPU ops	cuDNN, CUTLASS, FlashAttention
CUDA Runtime	GPU management, memory, stream	CUDA, ROCm, OpenCL
Driver / Hardware	Physical GPU execution	NVIDIA Driver, H100, A100

Inference Engine Comparison

Engine	Strengths	Best For
vLLM	PagedAttention, OpenAI-compati	General serving, research
SGLang	RadixAttention, structured gen	Agentic workloads, constrained
TensorRT-LLM	NVIDIA-optimized kernels, high	Production on NVIDIA hardware
llama.cpp	CPU+GPU, GGUF quantization, lo	Edge, local, consumer hardware
MLC LLM	Multi-platform (CUDA/Metal/Web	Cross-platform deployment

Key Concepts

PagedAttention:

KV cache stored in non-contiguous memory pages (like OS virtual memory). Eliminates fragmentation. Enables high batch sizes. Core of vLLM.

Continuous batching:

New requests inserted into batch as soon as a slot frees. Eliminates padding waste. Also called iteration-level scheduling.

CUDA graphs:

Capture GPU kernel sequence as a graph, replay with minimal CPU overhead. Reduces latency for fixed batch sizes.

FlashAttention:

IO-aware attention kernel. Fuses softmax + matmul. Avoids materializing $N \times N$ attention matrix. 2-4× faster, 5-20× less memory.

GGUF:

File format for quantized models (successor to GGML). Used by llama.cpp. Stores model weights + metadata in one file.

Model Format Checklist

- ☐ Safetensors: prefer over .bin for security + mmap support
- ☐ FP16 baseline: most inference engines expect fp16 weights
- ☐ GGUF: use for llama.cpp and consumer deployment
- ☐ TensorRT engine: pre-compile for fixed batch/seq shapes
- ☐ Verify config.json has correct rope_scaling for long context

Techniques: Optimization Deep Dives

Inference Engineering — inferenceengineering.tech

Quantization Formats

Format	Bits	VRAM vs FP16	Typical Quality Loss	Notes
FP32	32	2×	None (baseline)	Training only
FP16 / BF16	16	1×	None	Standard inference
FP8	8	0.5×	< 1%	H100/H200 native, NVIDIA Hoppe
INT8 (W8A8)	8	0.5×	< 1%	SmoothQuant, LLM.int8()
INT4 (W4A16)	4	0.25×	1–3%	GPTQ, AWQ, common for LLMs
INT4 (W4A8)	4/8	0.25×	1–3%	Higher throughput variant
2-bit	2	0.125×	5–10%+	QuIP#, AQLM — aggressive

KV Cache Formulas

KV cache per token (bytes):

$$2 \times n_layers \times (n_kv_heads \times d_head) \times dtype_bytes$$

Total KV cache (bytes):

$$kv_per_token \times max_seq_len \times batch_size$$

KV cache for Llama-3 70B (FP16):

$$2 \times 80 \times (8 \times 128) \times 2 = 327,680 \text{ bytes/token} \approx 320 \text{ KB/token}$$

KV cache compression (GQA ratio):

$$kv_size_reduction = n_kv_heads / n_query_heads$$

Parallelism Types

Type	What's Split	When to Use	Comm Overhead
Tensor (TP)	Weight matrices across GPUs	Model too large for 1 GPU	High (all-reduce each layer)
Pipeline (PP)	Layers across GPUs	Very deep models, large batch	Medium (bubble overhead)
Data (DP)	Batch across GPU replicas	High throughput, model fits in mem	Low (gradient sync only)
Sequence (SP)	Sequence length across GPUs	Very long context	Medium (ring attention)
Expert (EP)	MoE experts across GPUs	Large MoE models	Medium (all-to-all routing)

Speculative Decoding

Draft model:

Small fast model generates K candidate tokens speculatively (e.g., 4–8 tokens).

Target model:

Large model verifies all K drafts in a single parallel forward pass.

Acceptance rate ($\pm$):

Fraction of draft tokens accepted. Higher $\pm$ = better speedup. Depends on draft/target model quality.

Speedup:

$$\text{Expected tokens per step} = (1 - \pm^{(K+1)}) / (1 - \pm). \text{ Up to } 2\text{--}3\times \text{ at high acceptance rate.}$$

Modalities: Beyond Text

Inference Engineering — inferenceengineering.tech

Modality Pipelines

Modality	Input Format	Key Stage	Output Format
VLM	Image + text tokens	Vision encoder ! Text tokens	Text tokens ! L
ASR	Audio waveform (float32)	Encoder ! CTC or RNN	Text transcript
TTS	Text tokens	Acoustic model !	Audio waveform / MP3
Image Gen	Text prompt + noise latent	Diffusion UNet × N steps	Decoded image (JPEG/PNG)
Video Gen	Text/image prompt + noise	Spatiotemporal transformer	Frame sequence / MP4
Embeddings	Text / image / audio	Encoder forward pass only	Float vector (dense)

Vision Language Models

Vision encoder:

ViT (Vision Transformer) encodes image into patch embeddings. Typical: 336×336 px = 576 tokens.

Projector / adapter:

MLP or cross-attention layer maps vision embeddings into LLM token space.

Token budget:

Each image adds hundreds of tokens to context. High-res tiling multiplies cost further.

Dynamic resolution:

Encode image at multiple resolutions, select based on content. Used in InternVL, LLaVA-Next.

Diffusion Inference Levers

Inference steps:

Number of denoising iterations. Fewer steps = faster but lower quality. Range: 4 (LCM) to 50 (standard).

CFG scale:

Classifier-Free Guidance scale. Doubles compute (two forward passes). Range: 1–15.

Latent space:

Diffusion runs in compressed latent space (e.g., 64×64 for a 512×512 image). VAE decodes at the end.

Distilled models:

Smaller student models trained to match full model in fewer steps. SDXL-Turbo, FLUX-schnell.

Multimodal Serving Checklist

- ☐ Pre-encode images offline when possible — vision encoder is deterministic
- ☐ Cache vision embeddings by image hash
- ☐ Account for variable token counts in batching logic
- ☐ Use dynamic resolution only when image detail matters
- ☐ For diffusion: profile step count vs quality tradeoff on your prompts
- ☐ Run VAE decoder on separate stream to overlap with next request

Production: Autoscaling & Deployment

Inference Engineering — inferenceengineering.tech

Autoscaling Formulas

Minimum replicas:

$$\text{min_replicas} = \text{ceil}(\text{peak_rps} \times \text{avg_latency_s} / \text{requests_per_replica})$$

GPU count for throughput:

$$\text{gpus} = \text{ceil}(\text{target_tokens_per_sec} / \text{throughput_per_gpu})$$

Queue depth target:

Scale up when `queue_depth > scale_up_threshold` (typically 2–5 pending requests)

Scale-down delay:

Wait `cooldown_period` (60–300s) before removing replicas to avoid thrashing

Cold Start Mitigation Checklist

- ☐ Keep minimum 1 warm replica per deployment to absorb bursts
- ☐ Use container image pre-caching on all nodes (pull on startup)
- ☐ Store model weights on fast NVMe or in-memory (tmpfs) on node
- ☐ Use P2P weight loading — GPUs pull directly from NVMe, bypass CPU
- ☐ Implement readiness probes that wait until model is loaded
- ☐ Pre-warm KV cache with a dummy request after load
- ☐ Use spot/preemptible with fallback to on-demand for cost + availability

Batching Strategies

Strategy	How It Works	Best For
Static batching	Wait until <code>batch_size</code> requests	Offline, predictable workloads
Dynamic batching	Batch up to <code>max_size</code> or <code>max_wa</code>	Moderate traffic, mixed length
Continuous batching	Insert new requests mid-genera	Production LLM serving, high c
Chunked prefill	Split long prompts into chunks	Mixed long/short context, redu

Observability Key Metrics

TTFT P99:

Time to first token at 99th percentile. Primary SLO metric for interactive apps.

TPOT P50:

Median time per output token. Tracks streaming smoothness.

GPU memory utilization:

Target 80–90%. Below 70% = underutilized. Above 95% = risk of OOM.

Request queue depth:

Pending requests waiting for a slot. Alert if `> 10` for `> 30s`.

Token throughput:

Output tokens/second per GPU. Compare to theoretical max to measure efficiency.

KV cache hit rate:

Fraction of prefill tokens served from cache. Track for prompt caching systems.